

استخدم Git

لكي تسافر عبر الزمن والأبعاد

ما هو ال HEAD ؟ وما مفهوم ال Branch في Git ؟
هل فعلاً ال Git جهاز للسفر عبر الزمن والأبعاد؟



Ahmed El-Tabarani

Back-End Developer

أولاً لنتعرف على الـ HEAD الكبير

هو مؤشر يشارو على **commit**، وغالباً يكون على آخر **commit** في الـ **branch**، ويمكننا جعله يؤشر على أي **commit** آخر

ملحوظة:

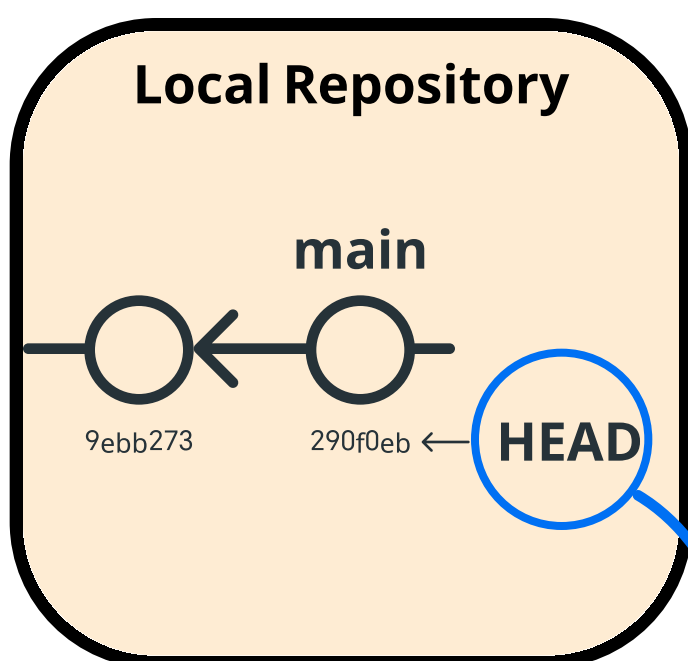
مكان الـ **HEAD** يعكس الملفات التي في الـ **Working Directory** فإذا غيرنا مكان الـ **HEAD** وجعلناه يشارو على **commit** قديم فإن محتوى الـ **Working Directory** ستكون نفس حالة المشروع التي كان عليها في هذا الـ **commit** القديم كما لو أن الـ **HEAD** أداة السفر عبر الزمن الخاصة بالـ **Git**

كيف نحرك مكان الـ HEAD ؟

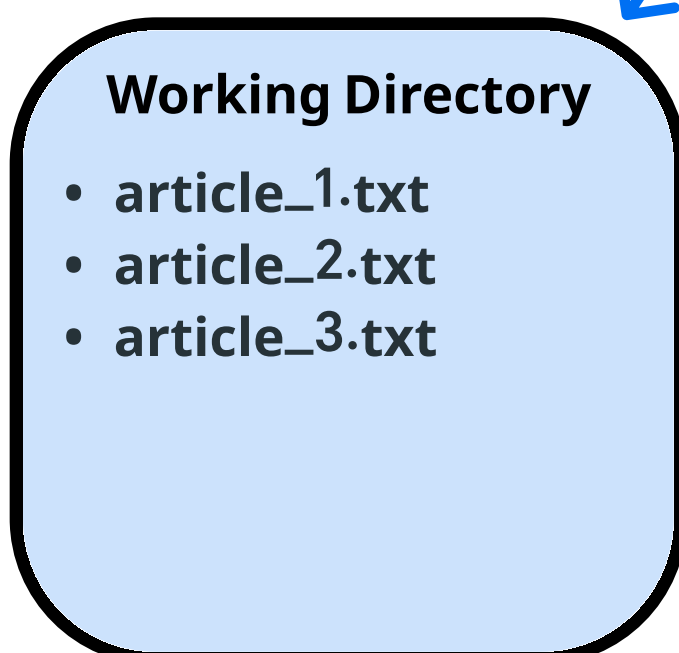
قبل أن نتعمق في عالم الـ **Branch** دعونا نستكشف كيف نسافر عبر الزمن عن طريق تحريك مكان الـ **HEAD** ونجعله يشارو على أي **commit** حتى وإن كان قديم

يوجد العديد من الأوامر التي نستطيع من خلالها تحريك الـ **HEAD**، ومن ضمنها:

git reset, git checkout, git switch



شكل الـ **Working Directory** هكذا عند الـ **commit** الـ **290f0eb**



Ahmed El-Tabarani
Back-End Developer



git reset

نستخدمه لتحريك كلا ال **HEAD** وال **branch** في آن واحد إلى ال **commit** الذي يتم تحديده

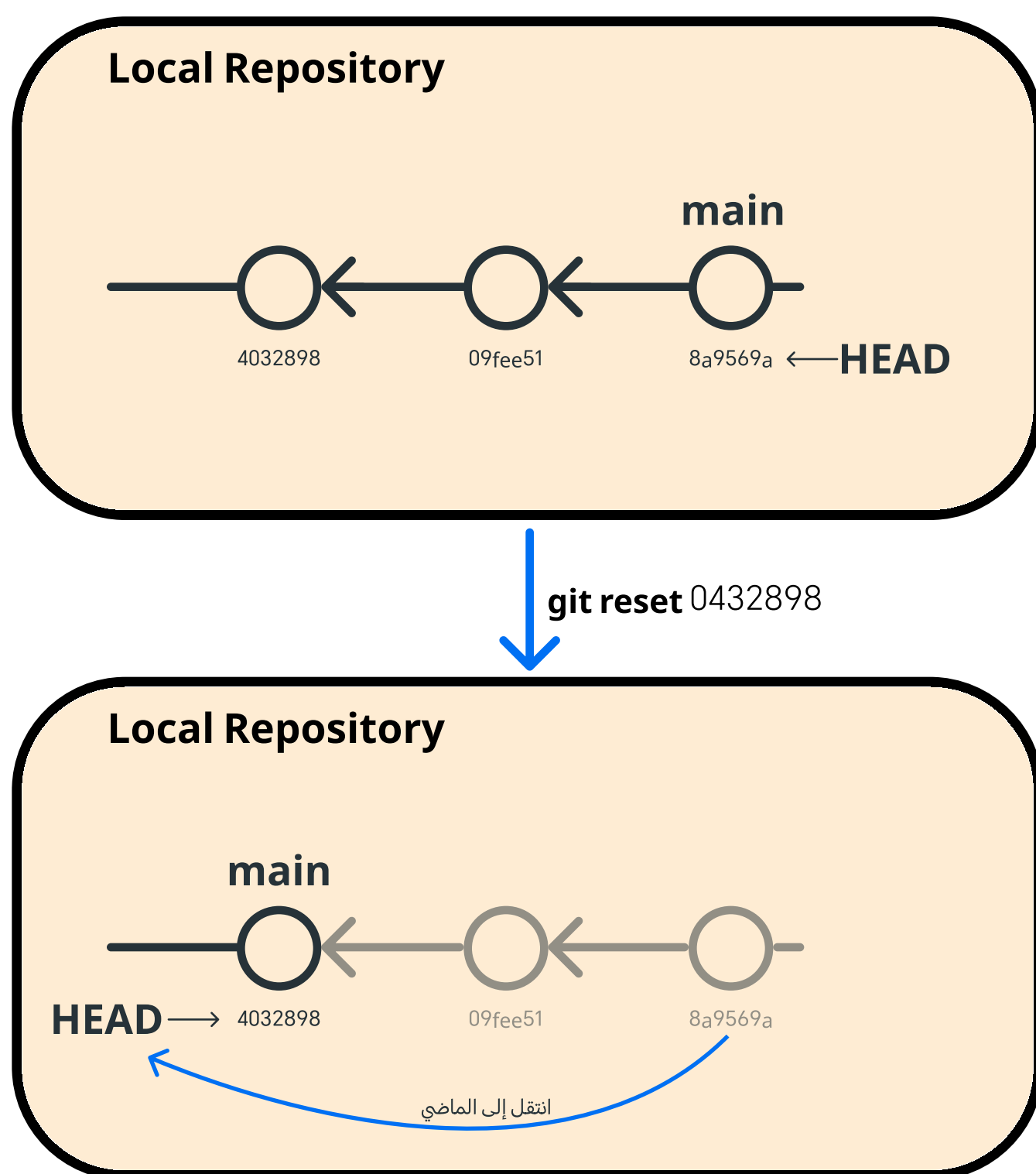
عليك أن تكون حذرًا في استخدام ال **git reset** لكي لا تفقد كل شيء

يمكنك استخدام **git reset HEAD~n** لتنتقل للماضي بعدد **n** من ال **commit**

وللعودة للـ **commit** الذي كنت فيه يمكنك استخدام الأمر **git reflog** لعرض كل ال **commit** ثم معرفة ال **commit** الذي كنت فيه

ثم استخدام **git reset** مجددًا للانتقال إليه

الم أقل لك أن ال **Git** يمتلك جهاز لسفر عبر الزمن



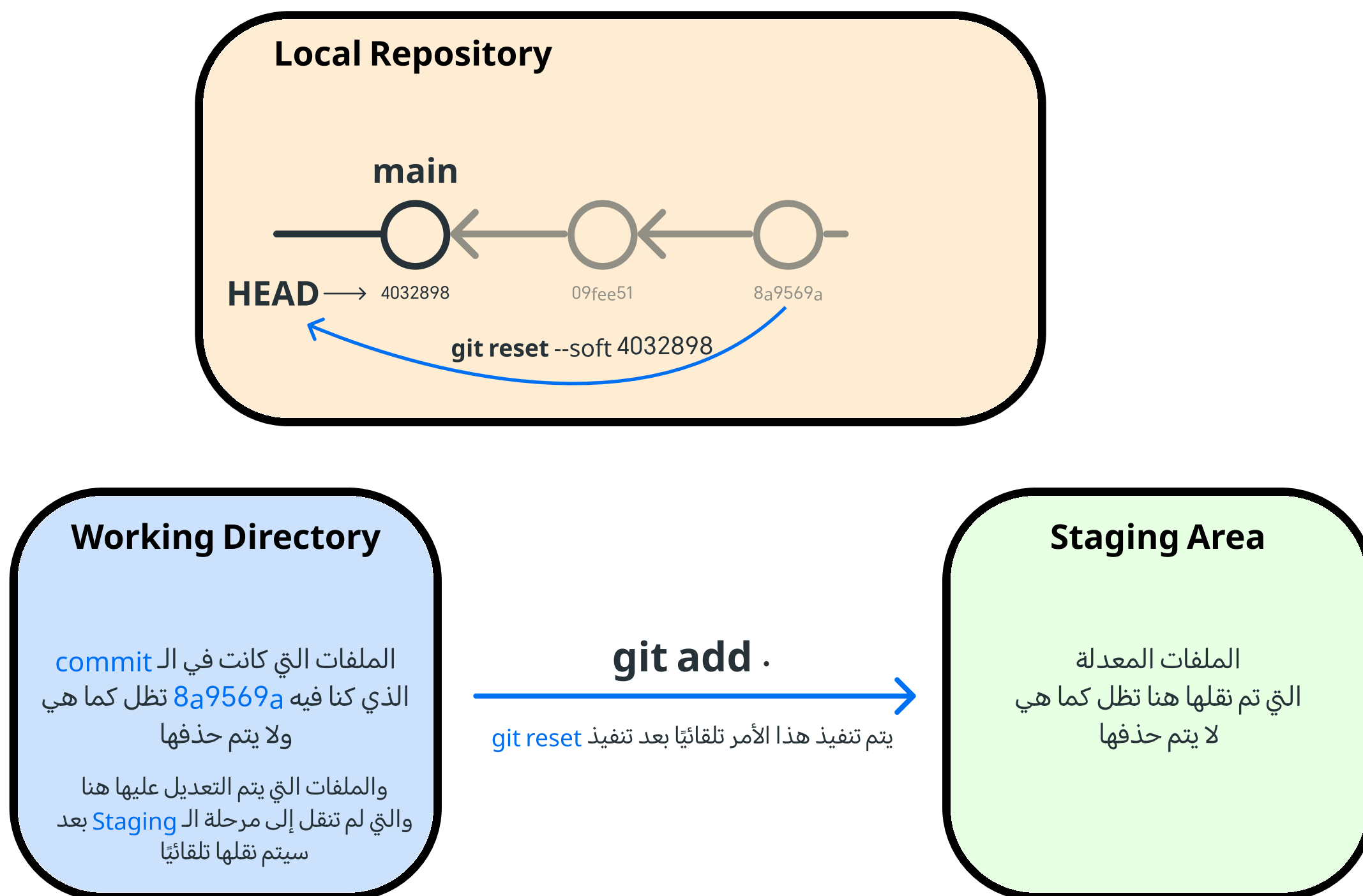
هناك ثلاث خيارات مهمة في ال **git reset** وهم **--soft** و **--mixed** و **--hard** وعليك أن تفرق بينهم وخصوصًا **--hard** لكي لا تفقد ما كل شيء



Ahmed El-Tabarani
Back-End Developer



git reset --soft



أهم النقاط:

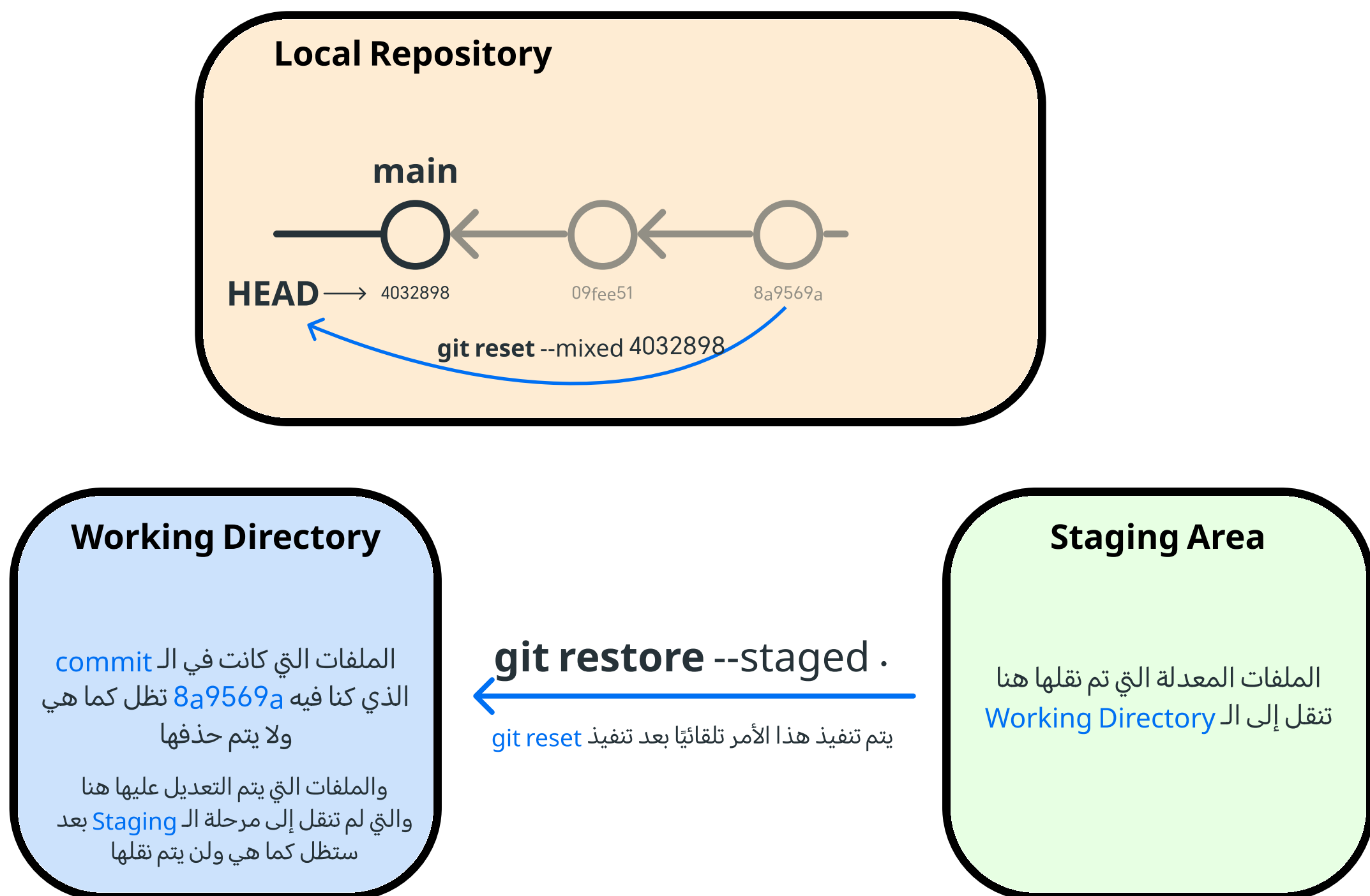
- لا يقوم بتعديل الـ **Working Directory** على الإطلاق بل يظل كما هو قبل الـ `git reset`
- لا يفعل أي شيء مع الملفات التي في مرحلة الـ **Staging** بل تظل كما هي
- التعديلات التي حدثت ما بين الـ **HEAD** والـ **commit** المحدد يتم نقلها إلى الـ **Staging**
- إذا كان هناك تعديلات كنا نعمل عليها في الـ **Working Directory** ولم تنقل إلى الـ **Staging** فسيتم نقلها تلقائيًا كما لو أنه قام بتنفيذ `git add`



Ahmed El-Tabarani
Back-End Developer



git reset --mixed (القيمة الافتراضية)



أهم النقاط:

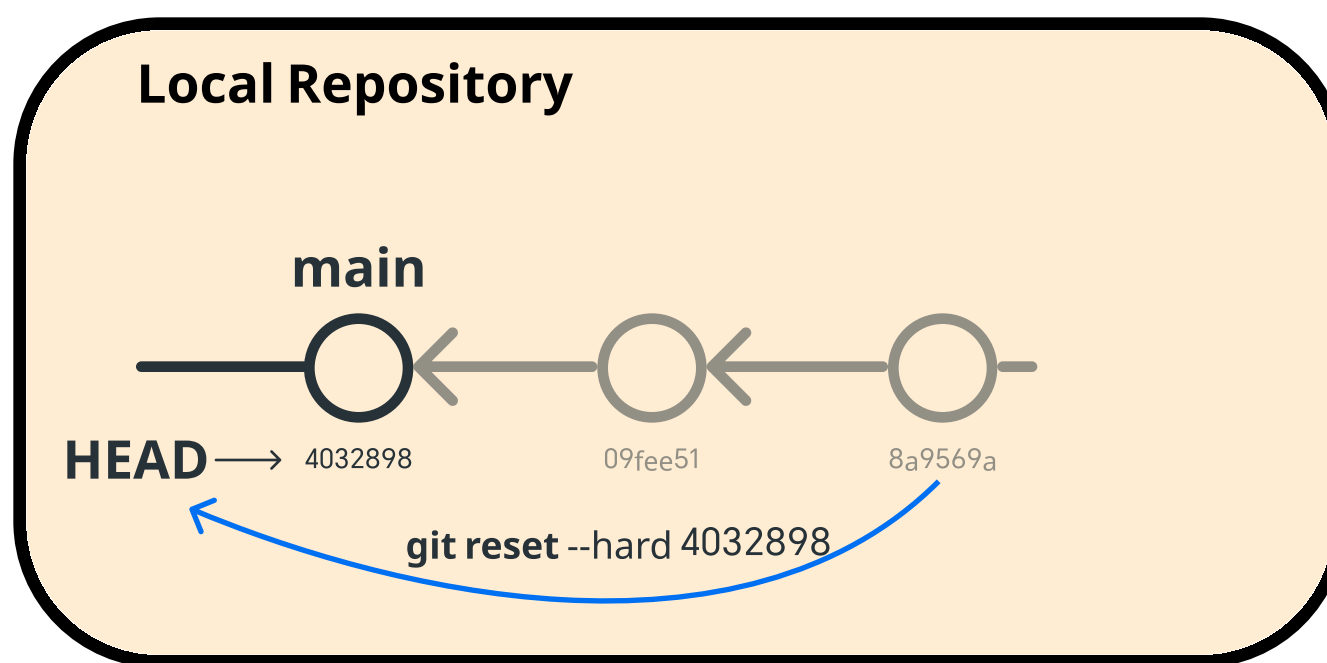
- لا يقوم بتعديل ال `Working Directory` على الإطلاق بل يظل كما هو قبل ال `git reset`
- كل الملفات التي في مرحلة ال `Staging` تخرج ويتم نقلها إلى ال `Working Directory` كما لو أنه قا بتنفيذ `git restore --staged`
- التعديلات التي حدثت ما بين ال `HEAD` و `commit` لا يتم نقلها إلى ال `Staging` بل تظل كما هي في ال `Working Directory`
- إذا قمنا بتنفيذ `--mixed` ثم قمنا بعمل `git add` فسنحصل على نفس النتيجة إذا نفذنا `--soft` من البداية



Ahmed El-Tabarani
Back-End Developer



git reset --hard (احذر أثناء استخدامه)



Working Directory

الملفات التي كانت في ال `commit` الذي كنا فيه `8a9569a` **يتم حذفها**

والملفات التي يتم التعديل عليها هنا والتي لم تنقل إلى مرحلة ال `Staging` بعد سيتم التخلص منها **دون رجعة**

Staging Area

الملفات المعدلة التي تم نقلها هنا يتم التخلص منها **دون رجعة**

احذر من استخدام `--hard` لأنه:

يتخلص من أي تعديلات تقوم بالعمل عليها سواء التعديلات التي في ال `Working Directory` أو التي نقلت إلى ال `Staging` **ستختفي دون رجعة**

أهم النقاط:

- يحرك ال `HEAD` إلى ال `commit` وينقل التعديلات إلى ال `Working Directory` مباشرة
- الملفات التي في ال `Working Directory` يتم استبدالها إجباريًا لجعل ال `Working Directory` يطابق النسخة التي كان عليها في ال `commit` المحدد
- يتخلص من أي تعديلات كانت في مرحلة ال `Staging` دون رجعة
- يتخلص من أي تعديلات كانت في ال `Working Directory` دون رجعة أي أنه يتخلص من ال `Modified Files`



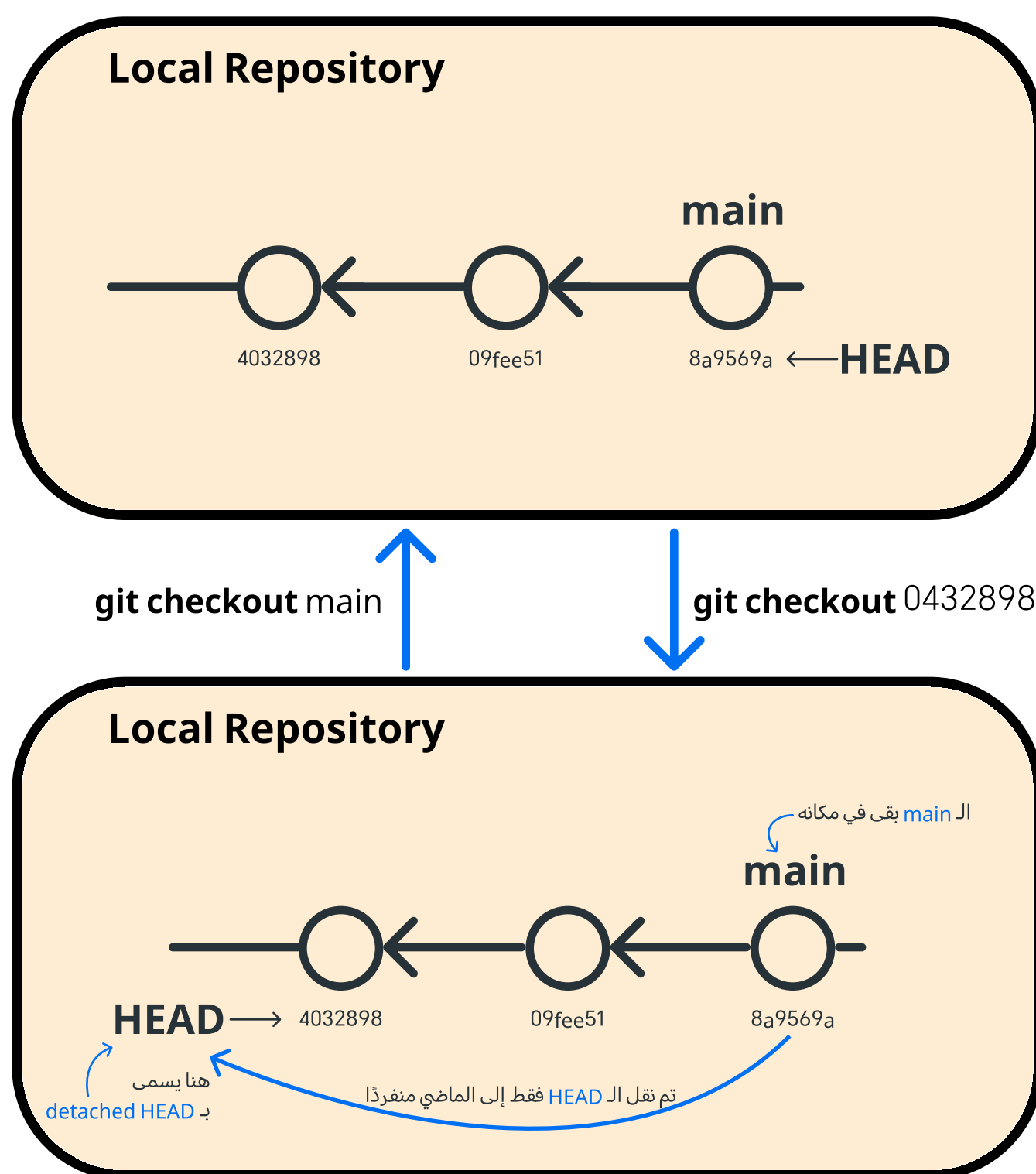
Ahmed El-Tabarani

Back-End Developer



git checkout

له استخدامات عديدة منها
تغيير مكان ال **HEAD** فقط إلى **commit** محدد



لاحظ التالي:

- ال **HEAD** هو من تحرك فقط والـ **main** بقى كما هو
- يمكننا جعل ال **HEAD** يرجع لأصله عن طريق **git checkout main**
- عندما يصبح ال **HEAD** يشارو على **commit** مجهول أي ليس هناك أي **branch** عليه ويسمى هنا **detached HEAD**
- في حالة ال **detached HEAD** إذا كنت تريد عمل تعديلات فيجب أن تنشئ **branch** في مكان ال **HEAD** ثم يمكنك عمل **commit** جديد يضم هذه التعديلات ويكون مرتبط بهذا ال **branch**

إذا كنت تملك ملفات تقوم بالتعديل عليها في ال **Working Directory** أو في ال **Staging**، فلن تستطيع تنفيذ ال **git checkout** والـ **Git** سينبهك بأنك لا يمكنك تحريك ال **HEAD** وهناك ملفات يتم التعديل عليها

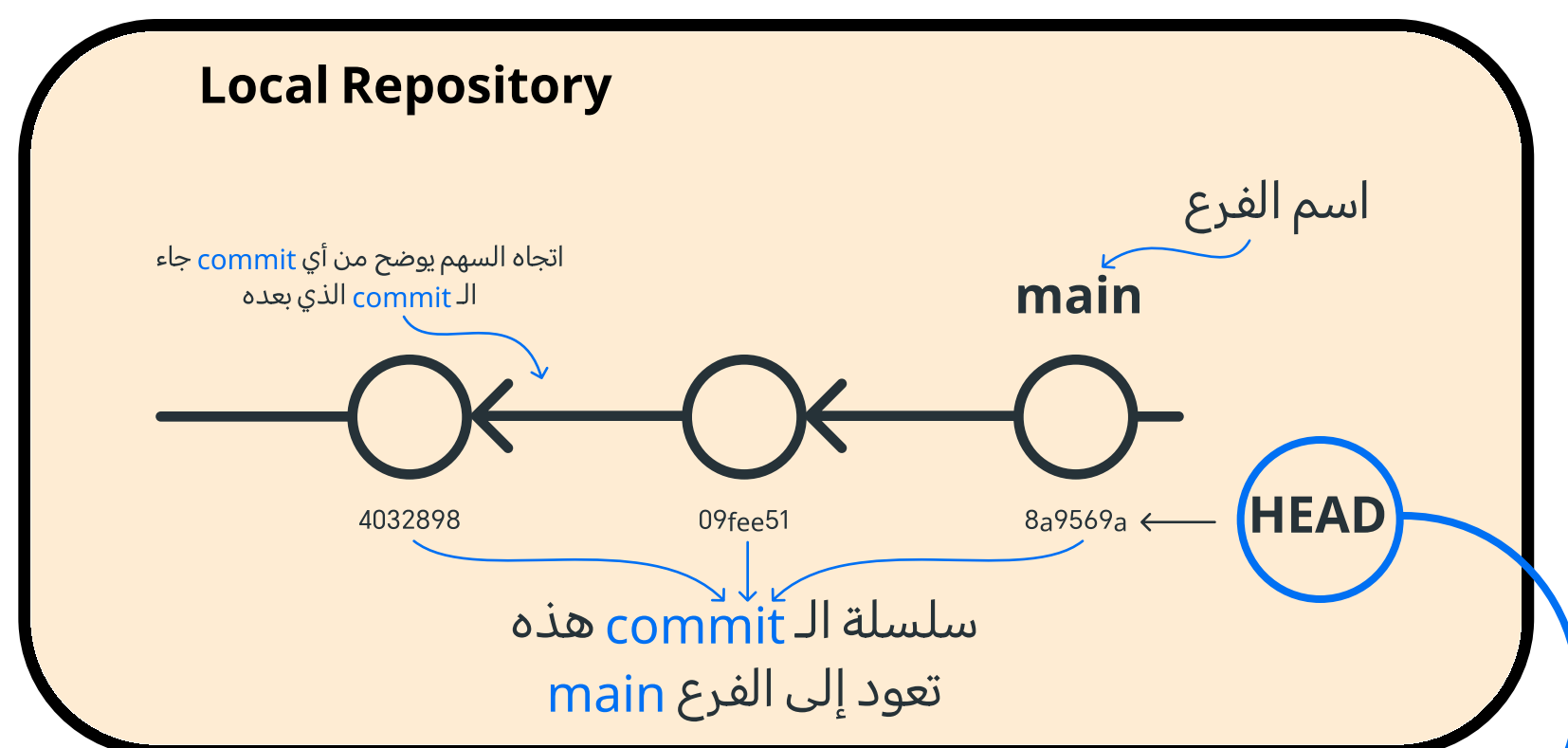


Ahmed El-Tabarani
Back-End Developer



ثانيًا ما هو الـ Branch؟

أهم المزايا التي يقدمها الـ **Git** هي فكرة الـ **branch** وهو تجميع أكثر من **commit** تحت مظلة واحدة ويستخدم الـ **HEAD** ليتنقل ما بينهم



ويمكنك إنشاء أكثر من **branch** للمشروع فعلى سبيل المثال فقط فرع خاص بالنسخة المستقرة للمشروع الخاصة بالـ **Production** فرع خاص فقط بالتطوير واختبار الأشياء فقط ويمكن لكل فريق أو شخص أن يكون ينشئ فرع خاص للشيء الذي يعمل عليه سواء إضافة شيء أو تعديل أو إصلاح مشكلة

Working Directory

- article_1.txt
- article_2.txt
- article_3.txt

والسبب أن الـ **branch** الواحد يكون مستقل تماما عن باقي الفروع الأخرى، فيمكنك إنشاء فرع وتلعب فيه وتختبر ثم تحذفه وكأنه لم يكن



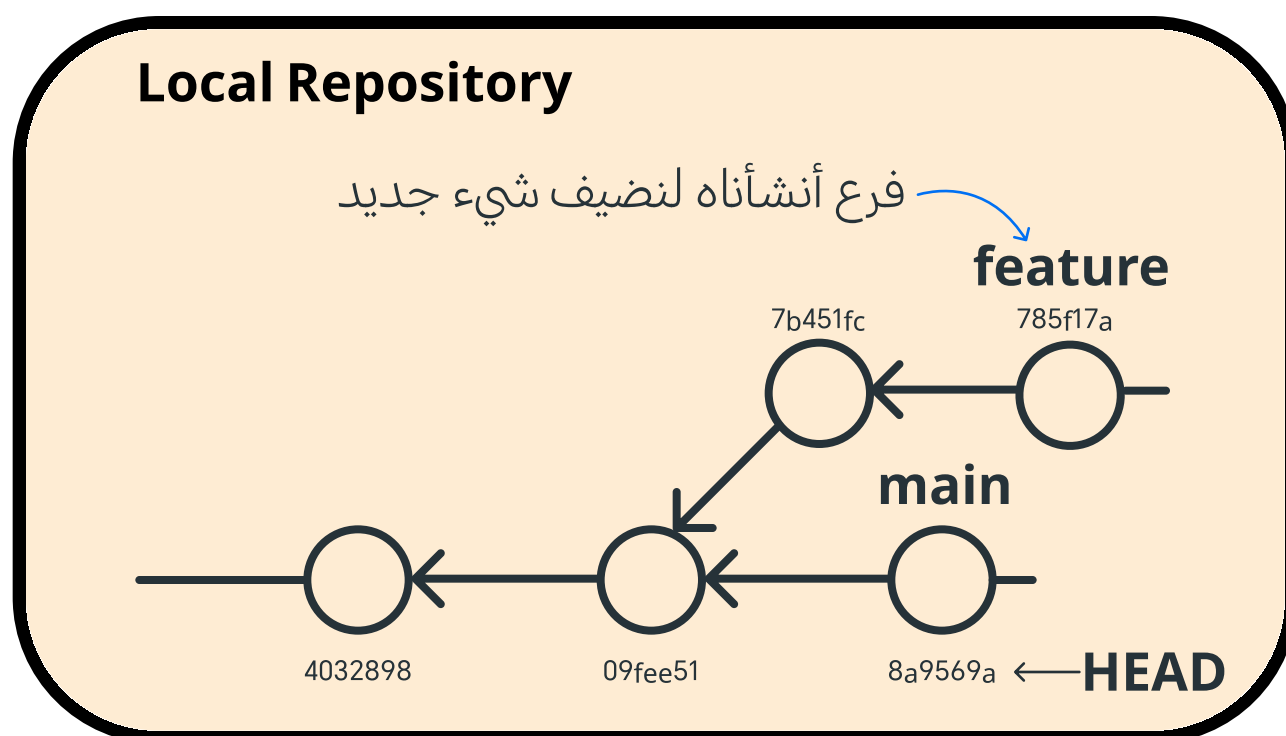
Ahmed El-Tabarani

Back-End Developer

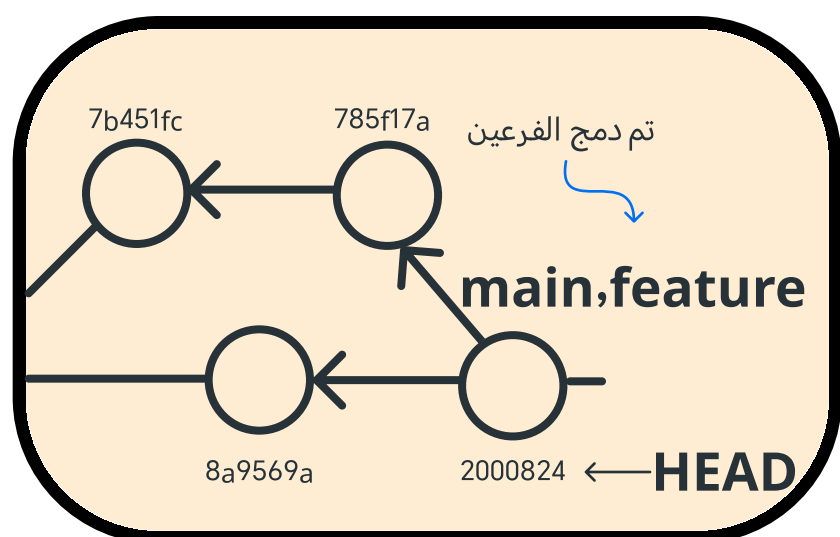


ما مميزات الأخرى في إنشاء أكثر من فرع ؟

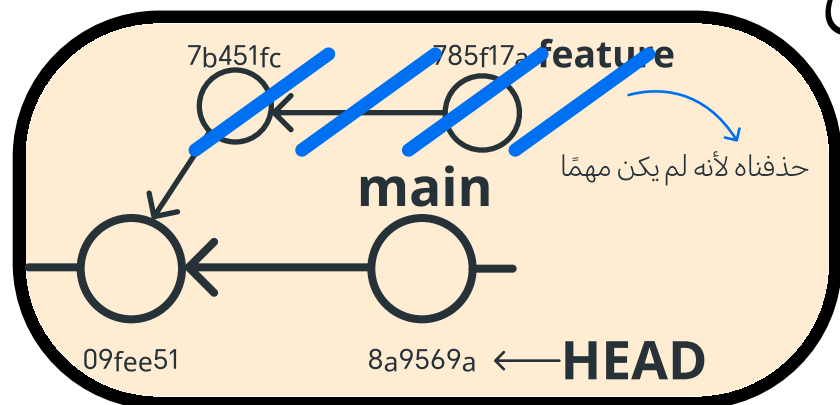
أولاً تقسيم العمل ضمن الفرق حيث يمكن لكل عضو أو كل فريق العمل على نسخته من المشروع في فرع مختلف ومستقل عن الآخرين



وعندما ينتهي أحد من عمله داخل نسخته من المشروع ويختبرها وتكون جاهزة



يمكننا أن ندمجها مع الفرع الرئيسي وتصبح جزءاً من المشروع



أو يمكنه حذف الـ **branch** وكان شيء لم يكن في حالة أنه كان فقط يختبر شيء ما أو أنه لم يعد مهماً للمشروع ولا يريده أحد

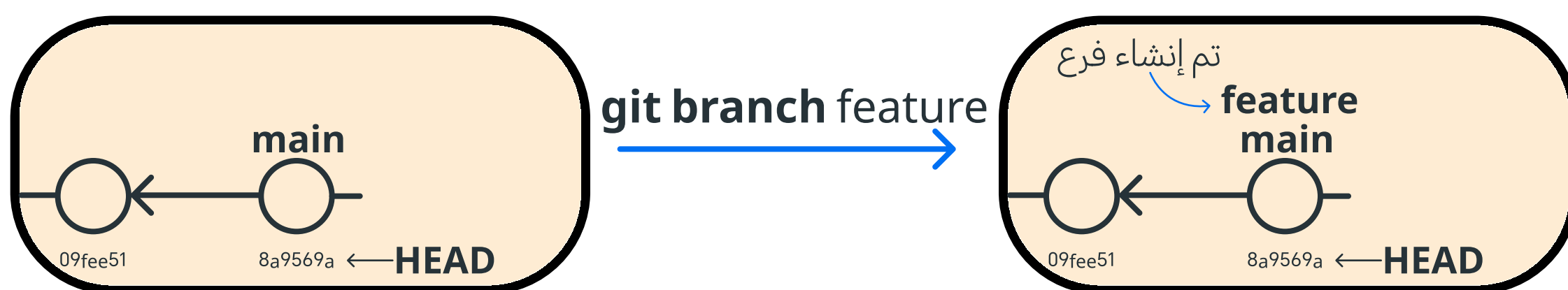


Ahmed El-Tabarani
Back-End Developer



git branch

الآن لنفترض أنني أريد أن أقوم بإضافة شيء ما في المشروع ولا أريد أن أعيث بالفرع الرئيسي هنا يمكننا أن ننشيء **branch** جديد، ونقوم بعمل ما نريده فيه



ونستطيع معرفة كل ال **branch** التي لدينا عن طريق **git branch -v**

```
> git branch -v
feature 8a9569a Revert "add xyz article"
* main 8a9569a Revert "add xyz article"
```

لاحظ أنه الآن أصبح لدينا فرع جديد وهو **feature** ووضعنا **-v** لكي نعرض آخر **commit** يقف عليه كل فرع وعلامة ***** تدل على مكان ال **HEAD** وتدل على الفرع الذي نقف فيه حالياً ونحن حالياً على **main**

وستلاحظ ان كلا من ال **main** و **feature** لديهم نفس ال **commit** وهذا يدل أن **feature** أشتق من ال **main** بالفعل



Ahmed El-Tabarani
Back-End Developer



git switch

الآن نريد أن نجعل الـ **HEAD** يترك الـ **main** ويذهب إلى الفرع الـ **feature**، ماذا نفعل؟ ببساطة لدينا أمر متخصص لهذا وهو **git switch**

```
> git switch feature  
Switched to branch 'feature'
```

الآن نقوم بتنفيذ **git branch** لتأكد فقط بأن الـ **HEAD** أم لا

```
> git branch  
* feature  
main
```

سترى أن * على **feature** لذا فإن الـ **HEAD** فعلاً تحرك إلى الـ **feature**

الآن ونحن في الـ **feature** لنقم بإضافة ملف جديد **article_2.txt** ثم نقوم بعمل **commit** لها لنرى ما الذي سيحدث



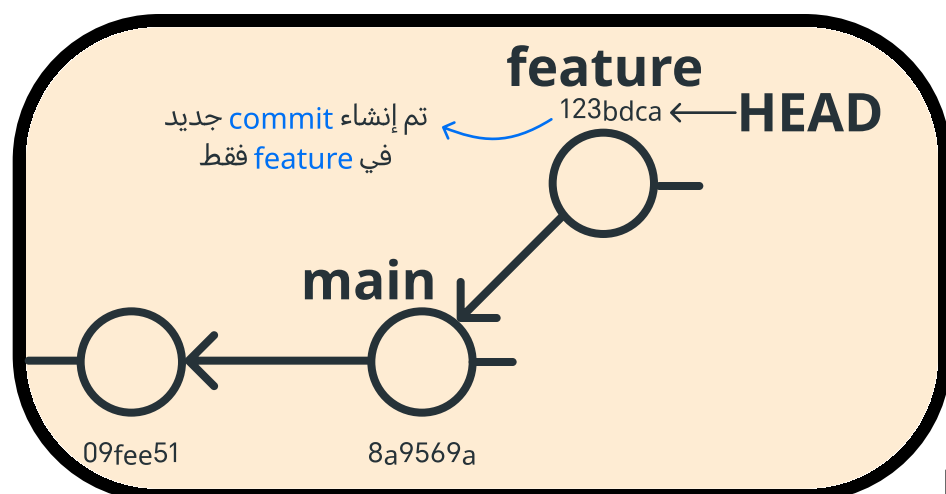
إنشاء commit جديد في feature

نُضيف ملف `article_2.txt` ثم ننفذ `git add`

```
> git add article_2.txt
```

ثم نقوم بتنفيذ `git commit`

```
> git commit -m "add article 2"
[feature 123bdca] add article 2
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 article_2.txt
```

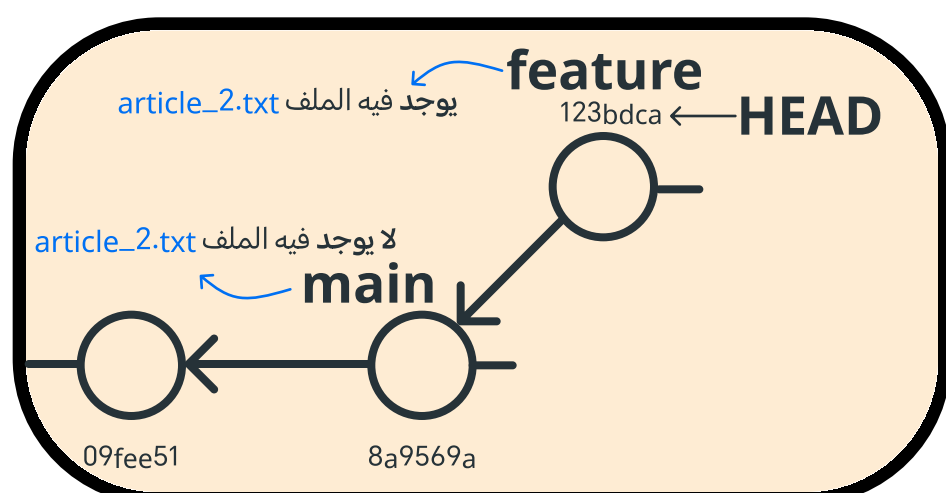


ال `feature` يقف على ال `commit` الجديد وال `main` مازال على القديم، لم يتأثر بأي شيء

لأن ما نقوم به في `feature` يكون منفصل تمامًا

عن ال `main`، ولاحظ أن ال `HEAD` أصبح في ال `feature`

ويشير على ال `commit` الذي فيه وليس الذي في ال `main`



الآن الملف الجديد `article_2.txt`

موجود فقط في `feature` وليس موجودًا في

ال `main`، وإن نقلنا ال `HEAD` إلى ال `main`

فلن نجد `article_2.txt`



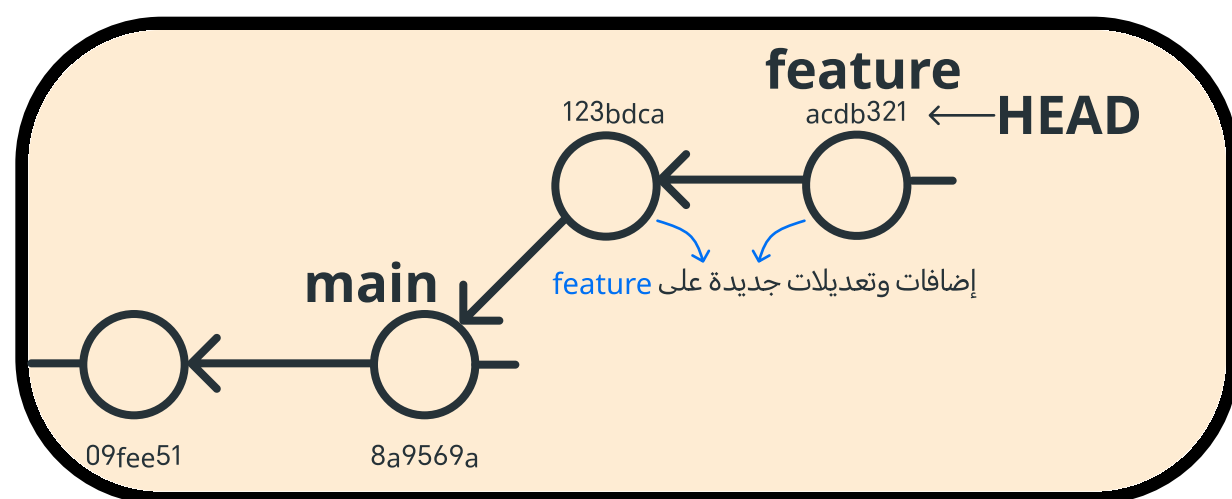
Ahmed El-Tabarani

Back-End Developer

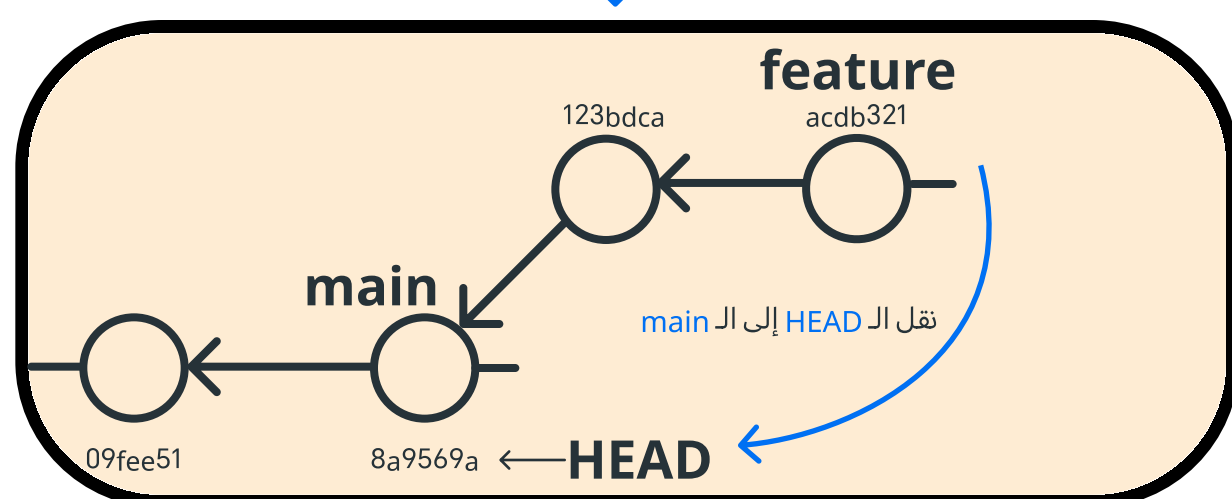


git merge

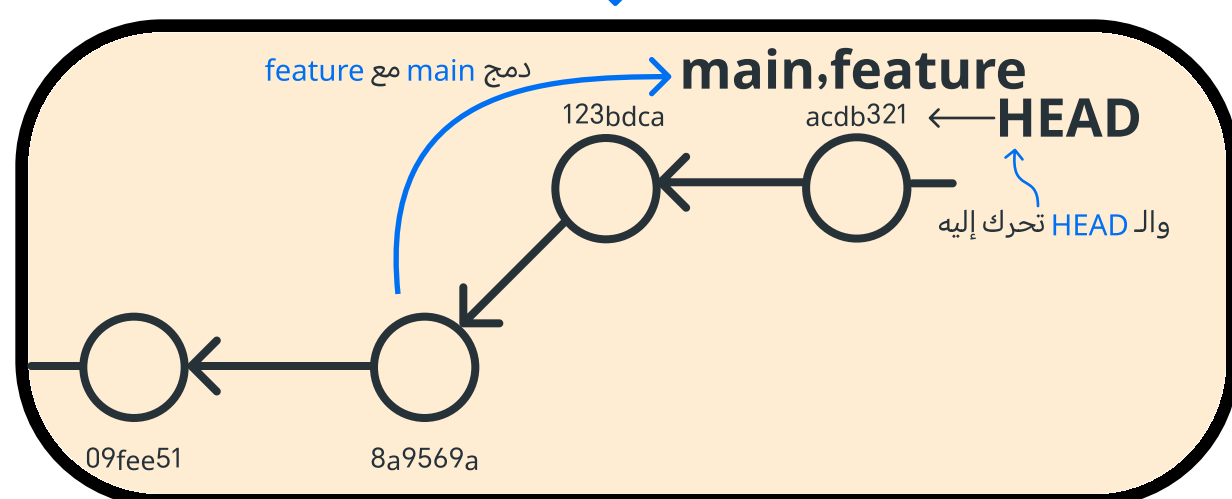
لنفترض أننا قمنا بعمل تعديلات كثيرة في الـ **feature** وتم اختبارها جيدًا وأصبح جاهزًا لوضع التعديلات التي فيه وندمجها في الـ **main**



`git switch main`



`git merge feature`

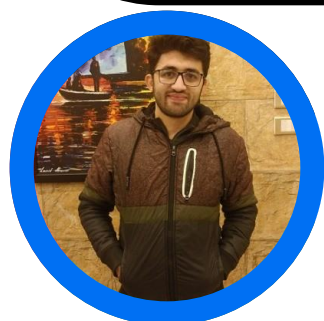


سنستخدم **git merge**، لكن أولًا علينا أن نذهب إلى الـ **main** عن طريق تنفيذ `git switch main`

بعد ما نقلنا الـ **HEAD** عند الـ **main** نقوم بتنفيذ أمر الدمج الجميل `git merge feture`

هكذا سيتم دمج ما في **feature** إلى الـ **main**

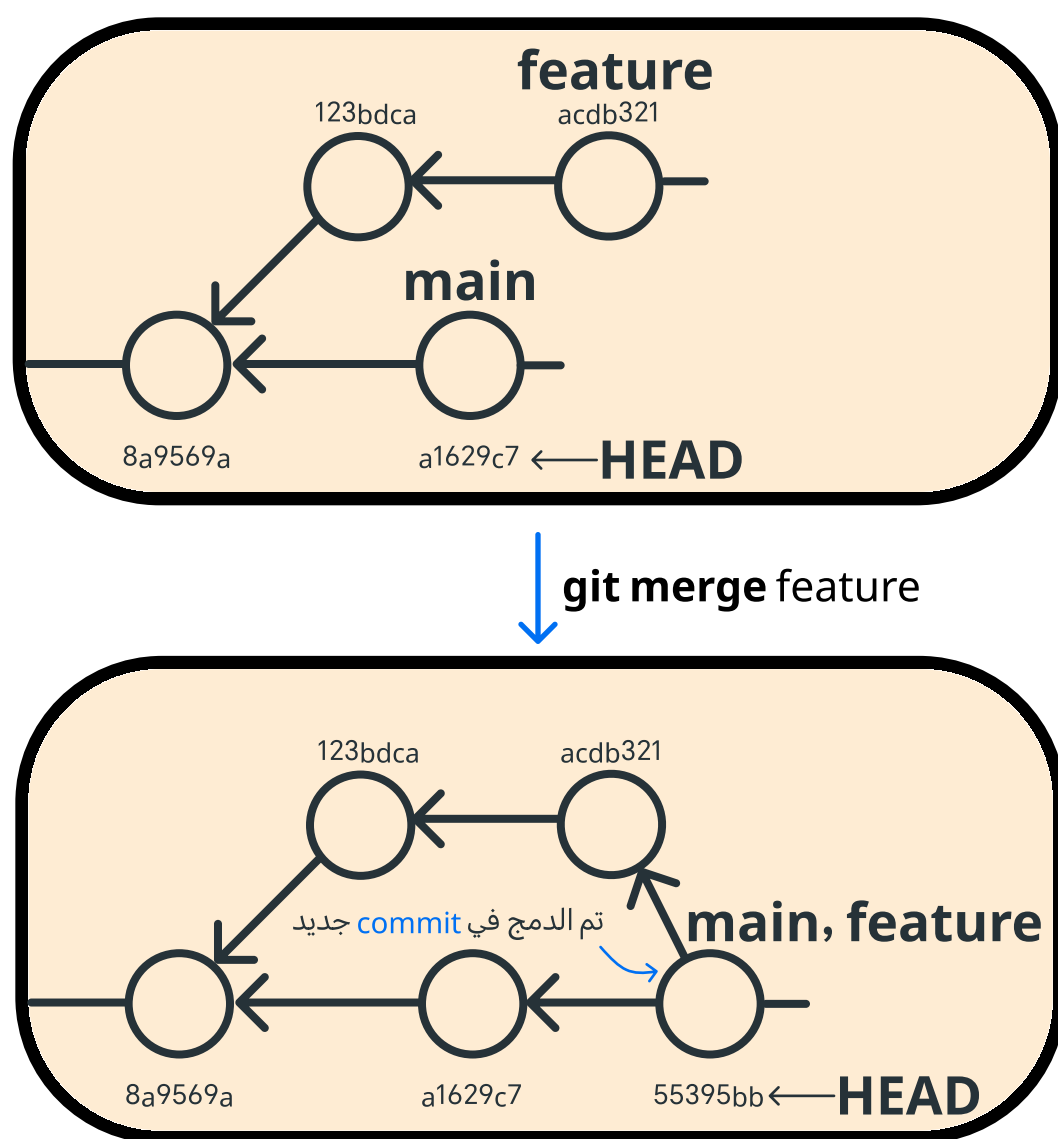
هذا النوع يسمى **fast-forward merge** لأنه فقط قام بتحريك الـ **main** إلى الـ **feature**



Ahmed El-Tabarani
Back-End Developer



ما فعلناه أشبه بأن الـ **main** ذهب إلى المستقبل حيث يوجد الـ **feature** لا أكثر، وهذا أبسط أنواع الدمج، ولكن الأمور بالطبع لا تكون دائمًا بسيطة



لاحظ أن هناك فرعين مختلفين يخرجان من نفس الـ **commit** الـ **8a9569a**

ولاحظ أن **feature** عنده **commit** ليس عند الـ **main** وأيضا الـ **main** عنده **commit** ليس عند الـ **feature**

في هذه الحالة عندما نحاول دمج **feature** مع **main** سيتم إنشاء **commit** جديد يضم التغيرات الموجودة في كلا الفرعين

هذا النوع يسمى عملية دمج الأبعاد الميتوفيزاقية .. أقصد **three way merge** لأنه يقوم بأخذ الـ **commit** من الـ **main** و **commit** من الـ **feature** ويدمجهما في **commit** جديد

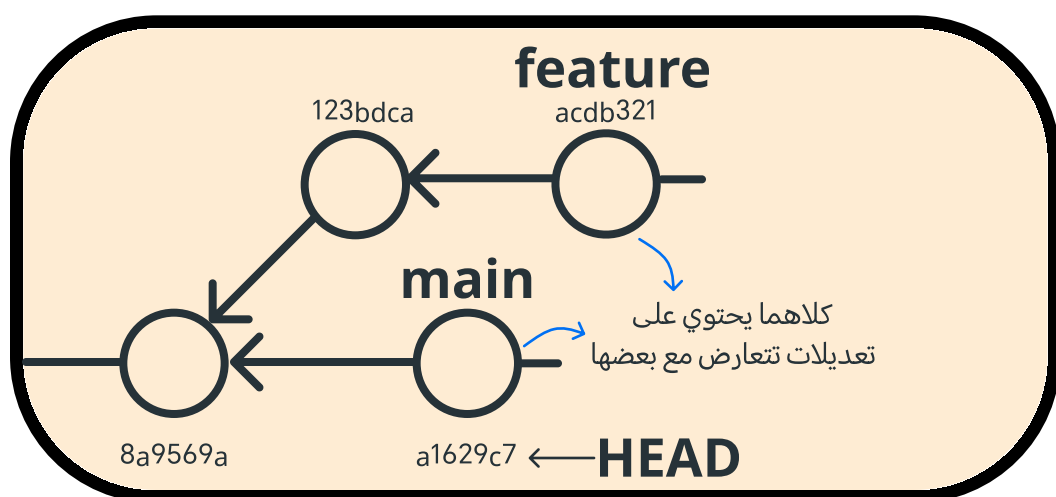


Ahmed El-Tabarani
Back-End Developer



مشكلة ال Merge Conflict

تخيل أنك في ال **feature** قمت بتعديل الملف **article_1.txt** ثم عملت **commit** ثم في ال **main** قمت بتعديل نفس الملف وعملت **commit**



هذه التعديلات قد تكون تتعارض مع بعضها البعض وهذا وارد وشائع جدًا

وعندما تحاول دمج التعديلات الموجودة

في ال **feature** وال **main**

ستظهر مشكلة التعارض

```
> git merge feature
Auto-merging article_1.txt
CONFLICT (content): Merge conflict in article_1.txt
Automatic merge failed; fix conflicts and then commit the result.
```

ستلاحظ أنه يخبرك بأن ال **Git** يختار ما بين التعديلات التي تتعارض مع بعضها

ولا يعرف هل يختار التي على ال **feature** أم التي على ال **main**

لذا يقول لك أن تحل المشكلة بنفسك بشكل يدوي ثم بعد انتهائك تقوم

بعمل **git add** للملفات التي أصلحت فيها التعارض ثم **git commit**



Ahmed El-Tabarani

Back-End Developer



عندما تفتح الملف التي به التعارض ستجد أن الـ **Git** يقوم بتقسيم الملف ويعرض لك التعديلات المتعارضة بهذا الشكل

```
<<<<<< HEAD
update article 1 in main
=====
update article 1 in feature
>>>>>> feature
```

ويقول لك **Git** أن القسم الذي في الأعلى هي التعديلات التي في الـ **HEAD** أي المكان الذي أنت فيه الآن وهو الـ **main**

والقسم الذي في الأسفل هي التعديلات التي كانت في الـ **feature**

بعد ما تقوم بحل كل التعارضات وتختار تقوم بعمل **git add** ثم **git commit**

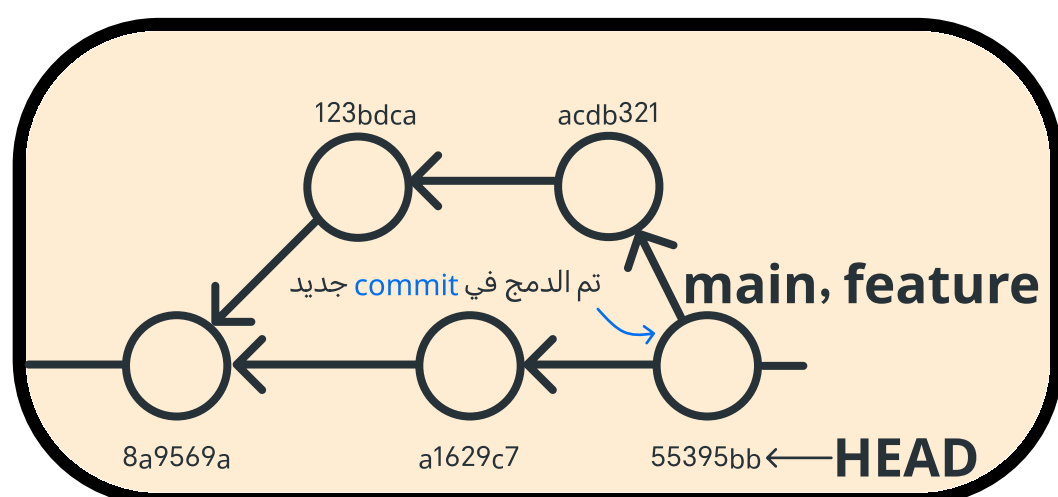
```
> git add article_.txt

> git commit -m "merge feature into main, and fix the merge conflict"
[main 55395bb] merge feature into main, and fix the merge conflict
```

↓ تم الدمج تلقائيًا بعد الـ **commit**

الآن تم حل التعارض بنجاح

وتم دمج الفرعين في **commit** جديد



مشكلة الـ **merge conflict** تحدث بكثرة

خصوصًا عندما تكون تتعامل مع **Remote Repository** ضمن فريق وحلها يتضمن تدخل الشخصين المسؤولين عن التعارض ليتم اختيار ومناقشة التعديلات المتعارضة بشكل يدوي



Ahmed El-Tabarani
Back-End Developer



خاتمة!

الآن أصبح لديك معرفة جيدة عن مفهوم

ال **Branch** في ال **Git**

حيث أنها تساعدنا عن إنشاء عدة نسخ من المشروع
تقسم العمل ضمن الفريق بشكل مستقل

بالطبع هذا مجرد ملخص بسيط لا أكثر
ويوجد مقالة على مدونتي استفيض قليلاً في الشرح
بشكل عملي، يمكنكم تفقد المقالة من هنا:

tabarani.tk/articles/git-with-branches



Ahmed El-Tabarani

Back-End Developer





أنا أحمد الطبراني، مهندس برمجيات SWE 😊

مبرمج متخصص في عالم ال Backend 🖥️⚙️

أحب دائمًا أن أشارك معرفتي المتواضعة مع الآخرين لعله عسى أن يستفيد شخص ما بما أكتبه 🙌
أكتب بعض المقالات عن أشياء مختلفة في عالم البرمجة، أرجوا أن تستفيدوا وتستمتعوا 😊

تابعني علي لينكدإن او علي مدونتي الشخصية